

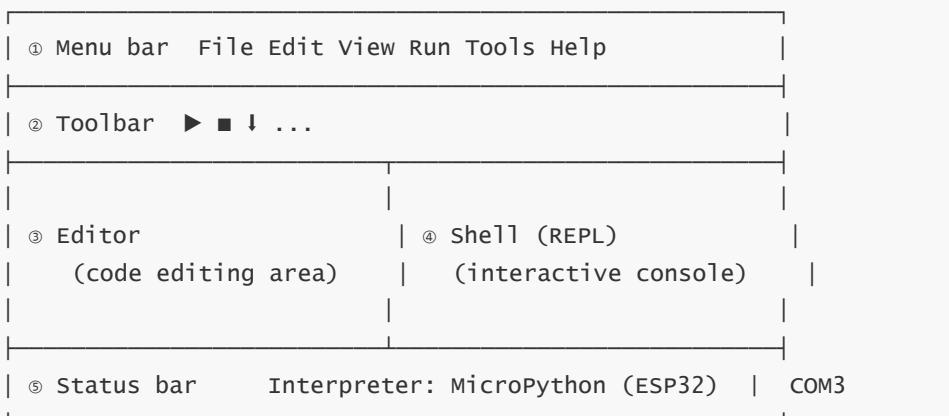
Thonny IDE Detailed Features

Thonny IDE Detailed Features

1. Interface Layout
 - 1.1 Open the File Management Panel
2. Editor Features
 - 2.1 Common Shortcuts
 - 2.2 Syntax Highlighting
 - 2.3 Code Completion
 - 2.4 Code Folding
3. REPL / Shell Usage Tips
 - 3.1 Basic Operations
 - 3.2 Tab Completion
 - 3.3 Command History
 - 3.4 Pasting Multi-line Code Blocks
 - 3.5 View Object Information
4. Interpreter Options and Settings
 - 4.1 Switching Interpreters
 - 4.2 Common Preferences
 - 4.3 MicroPython Connection Options
5. File Management
 - 5.1 File Operations Reference
 - 5.2 Startup Scripts
 - 5.3 File System Capacity
6. Common Problems

1. Interface Layout

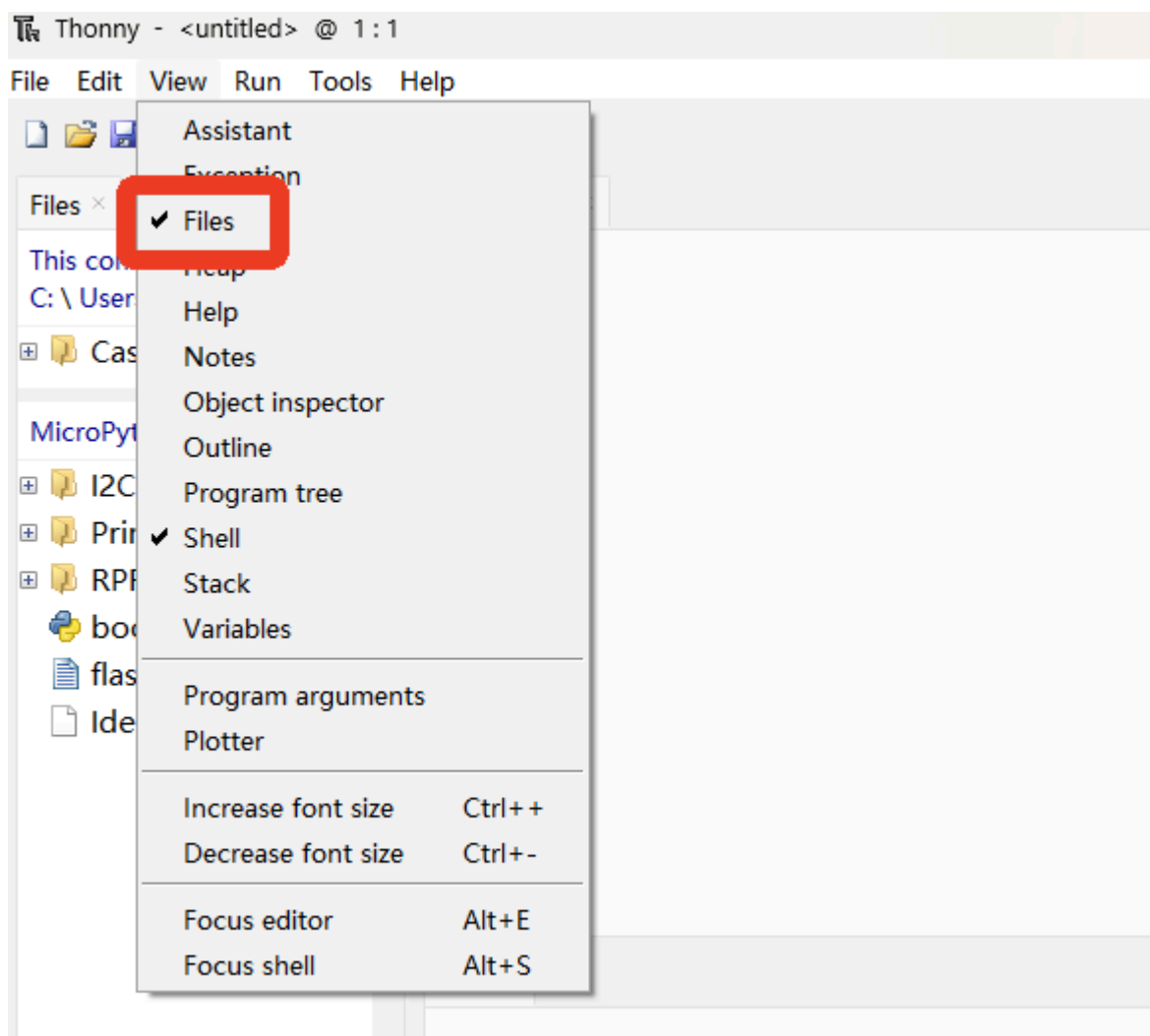
The Thonny interface is divided into five areas:

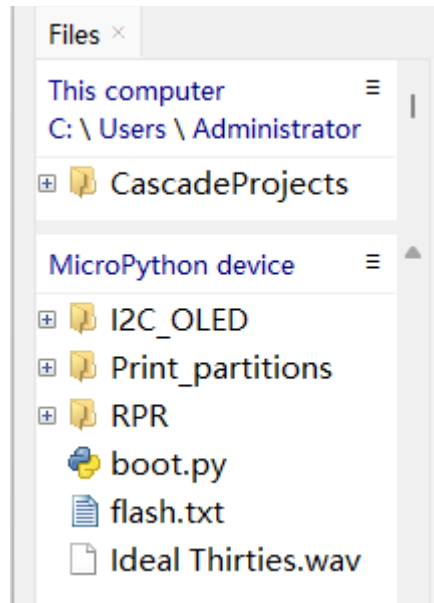


No.	Area	Description
①	Menu bar	File (file operations), Edit (editing operations), View (view toggles), Run (run/debug), Tools (tool menu), Help (help)
②	Toolbar	 Run script,  Stop program,  Step Into,  Step Over,  Step Out,  Resume execution
③	Editor	Multi-tab code editing area with syntax highlighting, line numbers, bracket matching, and auto-indentation
④	Shell	MicroPython REPL console for interactive Python execution; shows <code>print</code> output and exception messages
⑤	Status bar	Shows the line/column number on the left, and the current interpreter type and COM port on the right

1.1 Open the File Management Panel

Menu bar **View** → check **Files**, and a file panel appears on the left, divided into an upper and a lower part:





Panel	Path	Description
This computer	Local hard drive	Browse and select local files
MicroPython device	/flash	ESP32 internal file system

2. Editor Features

2.1 Common Shortcuts

Shortcut	Function
Ctrl + S	Save the current file
Ctrl + O	Open a file
Ctrl + N	Create a new file
Ctrl + /	Comment/uncomment
Ctrl + D	Duplicate the current line (or the selected region)
Ctrl + Z	Undo
Ctrl + Shift + Z	Redo
Tab	Indent (4 spaces)
Shift + Tab	Unindent
Ctrl + A	Select all
Ctrl + F	Find
Ctrl + H	Find and replace

Shortcut	Function
F5	Run the current script (upload to the device, then execute)
F6	Debug run (breakpoint mode)
F7	Step into a function
F8	Step over

2.2 Syntax Highlighting

Thonny automatically syntax-highlights Python code:

- **Purple** — keywords (`import`, `while`, `def`)
- **Blue** — built-in functions (`print`, `range`)
- **Green** — strings
- **Orange** — comments

2.3 Code Completion

Press `Ctrl + Space` while typing, or let the completion list pop up automatically; it contains module names, function names, and variable names.

Completion for MicroPython built-in modules (`machine`, `network`, `socket`, etc.) depends on whether Thonny is connected to the device; after connecting, it loads the device's module list.

2.4 Code Folding

In the line number area on the left of the editor, click `-` to fold function bodies / loop bodies / conditional branches, and click `+` to expand them.

3. REPL / Shell Usage Tips

3.1 Basic Operations

Operation	Key	Description
Interrupt a program	<code>Ctrl + C</code>	Stop the currently executing script and return to <code>>>></code>
Soft reset	<code>Ctrl + D</code>	Soft-reset the ESP32, equivalent to <code>machine.soft_reset()</code> , keeps the RAM contents
Clear Shell	<code>Ctrl + L</code>	Clear the Shell panel display (does not affect the device)
Paste multiple lines	<code>Ctrl + V</code>	Paste a multi-line code block directly; Thonny feeds it to the REPL line by line

3.2 Tab Completion

Type a partial name after `>>>` and press `Tab`:

```
>>> import machine
>>> machine.Pin.      # press Tab to pop up the completion list
OUT                IN                OPEN_DRAIN    PULL_UP
PULL_DOWN          IRQ_FALLING      IRQ_RISING    ...
>>> machine.Pin(48, machine.Pin.OUT) # continue typing after completion
```

3.3 Command History

Use the `↑` and `↓` keys to browse previously entered commands, avoiding retyping.

3.4 Pasting Multi-line Code Blocks

Paste multi-line code directly after `>>>`; Thonny automatically recognizes the indentation and executes it as a block:

```
>>> for i in range(3):
...     print(i)
...
0
1
2
```

3.5 View Object Information

```
>>> import machine
>>> dir(machine)      # list all attributes of the module
>>> help(machine.Pin) # view the help documentation for the Pin class
```

4. Interpreter Options and Settings

4.1 Switching Interpreters

Click the interpreter selection area in the bottom-right corner; a menu with several options pops up:

Interpreter	Applicable Scenario
MicroPython (ESP32)	Connect to the ESP32-S3 via the serial port
MicroPython (SSH)	Remote connection via Wi-Fi WebREPL
Local Python 3	Run local Python scripts on the computer
MicroPython (generic)	Connect to other MicroPython boards via the serial port

4.2 Common Preferences

Menu bar **Tools** → **Options**:

Tab	Recommended Setting
General	Language: Simplified Chinese or English; UI mode: regular
Editor	Font size 14; indent 4 spaces; show line numbers
Theme & Font	"Syntax theme" can be set to a dark theme (Dark)
Run & Debug	Uncheck "Interrupt working program on connect" (to avoid accidental interruption on connect)
Files	Default file path can be set to the local project directory

4.3 MicroPython Connection Options

Click **Install or update MicroPython** in the interpreter selection dialog to call esptool directly to flash the firmware (an alternative to the manual command line).

When connecting the ESP32 for the first time, the **Port** in the interpreter options must be manually set to the correct COM port. Thonny remembers the last selection and auto-connects next time.

5. File Management

5.1 File Operations Reference

Operation	Method
Upload to ESP32	Right-click a local file → Upload to /
Download to computer	Right-click a device file → Download to /
Delete	Right-click a file → Delete
Create a directory	Right-click blank space in the device panel → New directory
Rename	Select the file → F2 (or right-click → Rename)

5.2 Startup Scripts

Execution order after the ESP32 is powered on: after the `MicroPython` firmware is flashed, the board automatically looks for two scripts on power-up, running `boot.py` first and then `main.py`.

As the initialization script, `boot.py` **fully resets the Wi-Fi adapter first to avoid state issues from a soft restart, then blocks and waits until it connects to the router.**

Only after the network is established does the system run `main.py`, which is only responsible for importing business modules and starting the application.

There is a **risk** to note: whether in boot or main, if an infinite loop or an exception hangs the program, you can quickly press `Ctrl+C` at the moment of power-up to interrupt the script and enter the command-line `REPL` to fix the code.

```
# boot.py – runs first on boot
import network, time

wlan = network.WLAN(network.STA_IF)
wlan.active(False); time.sleep_ms(500)
wlan.active(True); time.sleep_ms(200)
wlan.connect("SSID", "PASSWORD")
while not wlan.isconnected():
    time.sleep(0.5)
print("Wi-Fi OK:", wlan.ifconfig())
```

```
# main.py – runs after boot.py
import my_app

my_app.run()
```

The firmware is just the MicroPython runtime environment; these two startup scripts must be **manually uploaded to the ESP32 file system**.

After the board powers on, the kernel automatically searches for these two files; if found, it runs them in sequence; if not, it directly enters the serial-interactive `REPL`.

5.3 File System Capacity

```
>>> import os
>>> fs_stat = os.statvfs('/flash')
>>> fs_stat[0] * fs_stat[3] // 1024    # free space (KB)
>>> fs_stat[0] * fs_stat[2] // 1024    # total space (KB)
```

```
>>> import os
>>> fs_stat = os.statvfs('/flash')
>>> fs_stat[0] * fs_stat[3] // 1024    # 剩余空间 (KB)
11688
>>> fs_stat[0] * fs_stat[2] // 1024    # 总空间 (KB)
14336
```

The MicroPython firmware takes up about 1~2MB; the rest of the Flash is available as the file system.

6. Common Problems

Symptom	Cause	Solution
No <code>>>></code> prompt in Shell	Interpreter not connected correctly	Make sure MicroPython (ESP32) is selected at the bottom-right and the correct COM port is chosen
Running a script shows <code>save file</code> , then nothing happens	The file cannot be written to the device	Check whether the device file system is full; or use <code>Ctrl + C</code> in the REPL to interrupt a stuck program
IOError when saving a file to the device from the editor	Flash file system corrupted	Re-flash the firmware, or format with <code>import os; os.vfsLfs2.mkfs(bdev)</code>
Chinese file names display as garbled in the device panel	MicroPython file names only support ASCII	Change the file name to English or pinyin
The device program keeps running after closing Thonny	MicroPython executes continuously on power-up; disconnecting the IDE does not affect it	Normal behavior; interrupt with <code>Ctrl + C</code> or reset with EN
Existing files not visible in the device panel	Thonny not refreshed	Right-click blank space in the device panel → Refresh
Not sure which file a multi-file project starts from	The device boots in the <code>boot.py</code> → <code>main.py</code> order	<code>import</code> other modules in <code>main.py</code>